Fix Report

Overview

The evolution from the initial implementation to the latest version transforms a basic Hybrid FTP system (TCP Control Channel + UDP RDT Data Channel) into a robust, secure, and fully standards-compliant implementation.
Below is a detailed report on the changes made to both the Client and Server codebases and the specific issues, bugs, and security vulnerabilities those changes resolve.

Client-Side Changes & Resolved Issues

1. Fixed Automatic Deletion of Downloaded Files

- **Changes:** In the original client (`rdt_download`), the variable `success` was initialized to `False` and never updated to `True` upon successful file completion. In the latest client, `success = True` is set when the `FLAG_FIN` packet is received.
- **Issue Resolved:** In the old version, the `finally` block always evaluated `if not success:` as `True`, immediately deleting every successfully downloaded file after completion. The update ensures downloaded files are retained.

2. Full Active Mode (`PORT`) Support

- **Changes:** Added `active_mode` tracking, socket binding on `0.0.0.0:<data_port>`, and a parameter `is_active_mode` in `rdt_upload`. When uploading in Active Mode, the client waits for an initial UDP ping/SYN packet from the server before commencing data transmission.
- **Issue Resolved:** The old client only supported Passive Mode (`PASV`) via macro commands (`GET`, `PUT`, `LIST`). The updated client allows standard `PORT` negotiations for Active Mode data channels.

3. Graceful User Interrupts & Asynchronous Abort (`ABOR`)

- **Changes:** Added `try...except KeyboardInterrupt` blocks inside `rdt_download` and `rdt_upload`. When a user presses `Ctrl+C` during a transfer, the client catches the exception and immediately transmits an `ABOR` command to the server over the TCP control socket.
- **Issue Resolved:** Previously, interrupting a transfer with `Ctrl+C` crashed the client process locally while leaving the server indefinitely waiting for UDP packets or timing out.

4. Direct FTP Standard Command Processing in CLI

- **Changes:** Expanded the main CLI loop to handle standard FTP commands (`RETR`, `STOR`, `APPE`, `STOU`, `NLST`, `PASV`, `PORT`) directly alongside shortcut macros (`GET`, `PUT`, `LIST`).
- **Issue Resolved:** In the old version, entering standard FTP commands like `RETR file.txt` sent the TCP request to the server, but the client never launched its local UDP RDT

receiving or sending routines (rdt_download/rdt_upload), resulting in hung data channels.

Server-Side Changes & Resolved Issues

1. Sandboxing & Path Traversal Prevention

- **Changes:** Introduced self.base_dir = os.path.abspath(os.getcwd()) and added access verification checks (if not target_path.startswith(self.base_dir):) across all directory and file commands (CWD, MKD, RMD, LIST, NLST, RETR, STOR, DELE, RNFR, RNTO, APPE, MDTM).
- **Issue Resolved:** The old server used os.path.abspath without checking whether target paths breached the server's root directory. Users could execute commands like CDUP or pass ../../ paths to escape the restricted directory and access system files.

2. Non-Blocking Asynchronous Abort via Socket Multiplexing (select.select)

- **Changes:** Replaced plain recvfrom socket calls in rdt_send and rdt_recv with I/O multiplexing: select.select([self.control_sock, self.data_sock], [], [], timeout).
- **Issue Resolved:** Previously, while transferring UDP data, the server thread was blocked listening *only* on the UDP socket. If a client sent an ABOR command over TCP, the server ignored it until the entire UDP transfer finished or timed out. With select, the server monitors both sockets simultaneously and cancels the UDP stream immediately upon receiving an ABOR command.

3. Active Mode UDP Connection Initiation

- **Changes:** In handle_port, the client's IP and Port are stored as self.client_data_ip and self.client_data_port. During Active Mode uploads (STOR, APPE, STOU), the server fires an initial UDP FLAG_SYN packet to the client's listening UDP socket prior to starting rdt_recv.
- **Issue Resolved:** Solved the chicken-and-egg connection issue where the client waited for the server's UDP ping in Active Mode while the server was simultaneously waiting for client data without initiating contact.

4. Data Socket Readiness Validation

- **Changes:** Added pre-transfer checks (if not self.data_sock: self.send_response("425 Use PORT or PASV first.\r\n")) in all data transfer commands (RETR, STOR, APPE, STOU, LIST, NLST).
- **Issue Resolved:** Prevents potential unhandled exceptions or crashes on the server if a client attempts data transfers without first negotiating PORT or PASV.

5. Rename State Clean-up & Formatting Improvements

- **Changes:**
    - Resets self.rename_from_path = None whenever any command other than RNTO is issued.

    - Enhanced `handle_list` output to include standard UNIX permission strings (`drwxr-xr-x`, `-rw-r--r--`) and modification timestamps (`datetime.datetime.fromtimestamp(mtime).strftime('%b %d %H:%M')`).
- **Issue Resolved:** Fixed stale rename states if a user executed `RNFR` followed by an unrelated command, and provided client applications with UNIX-compliant file listings.

Comparison Summary Table

Feature / Area	Old Files (Source 14 & 15)	Latest Files (Source 16 & 17)	Primary Benefit / Issue Resolved
Downloaded Files	Deleted upon completion due to `success` flag bug.	Retained correctly upon receiving `FLAG_FIN`.	Fixes file deletion bug after download.
Directory Security	Allowed path traversal via `..` or `CDUP`.	Restricted to `base_dir` sandbox.	Prevents unauthorized file access.
Abort Operations	Blocked during UDP streams; `ABOR` delayed.	Non-blocking socket multiplexing via `select`.	Instant cancellation of active transfers.
Data Modes	Passive mode (`PASV`) only.	Full Active (`PORT`) & Passive (`PASV`) support.	FTP standards compliance.
CLI Handling	Macros only (`GET`, `PUT`, `LIST`).	Supports raw FTP commands (`RETR`, `STOR`, `STOU`, `APPE`, etc.).	Improved client usability & protocol compliance.